

Relatório 3: Operações de Ponto Flutuante

João Bruno dos Santos, João Lucas Medina Kormann

Universidade Federal de Santa Catarina

I. INTRODUÇÃO

Este relatório tem o objetivo de apresentar a metodologia aplicada para a resolução dos problemas do Laboratório 3 da disciplina Organização de Computadores I, ministrada pelo professor Marcelo Daniel Berejuck, utilizando a linguagem Assembly para a arquitetura MIPS (Microprocessor without Interlocked Pipelined Stages) e o simulador MARS (MIPS Assembler and Runtime Simulator). O trabalho consiste no desenvolvimento de um programa capaz de calcular o valor aproximado da raiz quadrada de um número através do método iterativo de Newton além também de calcular uma aproximação para a função seno de um valor.

II. IMPLEMENTAÇÃO DO MÉTODO DE NEWTON

Para a implementação do método iterativo de Newton, seguimos o processo descrito no laboratório: $Estimativa = \frac{(\frac{x}{Estimativa}) + Estimativa}{2}$ onde temos como dados salvos na memória x , o número em questão, n , o número de repetições do método, $estimativa$, dado que será iterado e modificado várias vezes junto de dois espaços de texto para auxiliarem o print. Com estes dados iniciados, começamos o main com a instrução *jal ler_terminal* que pula até a função designada para leitura do teclado.

```
.data
x:      .double 293
estimativa: .double 1
n:      .word 10

espaco:  .ascii " "
chamada: .ascii "Insira o numero de iteracoes: "

.text
main:
    jal  ler_terminal
```

Figura 1 - Inicialização método de Newton

A. Procedimento de leitura - Ler_terminal

De forma simples, carregamos 4 em \$v0 para imprimir um inteiro na tela, carregando então o conteúdo de “chamada” em \$a0 para selecioná-la a aparecer no terminal. Após isso carregamos 5 em \$v0 para ler o inteiro escrito no terminal pelo usuário, salvando em n o número obtido e, por fim, carregamos em \$a2 o valor de n junto a um *jr \$ra* para retornar à função de onde ela foi chamada.

```
ler_terminal:
    li    $v0, 4          # Print String
    la    $a0, chamada
    syscall

    li    $v0, 5          # Ler inteiro
    syscall
    sw    $v0, n          # Guarda ele em n

    la    $a2, n          # Salva em a2
    jr    $ra
```

Figura 2 - Ler_terminal

Retornando ao main, carregamos \$a0 com o valor inicial da estimativa (1) enquanto carregamos \$a1 com o valor de x (número que queremos saber a raiz). Logo após, saltamos com um *jal raiz_quadrada_newton*.

```
la    $a0, estimativa
la    $a1, x

jal    raiz_quadrada_newton
```

Figura 3 - Continuação main

B. Cálculo raiz - raiz_quadrada_newton

Para o cálculo da raiz quadrada com newton, iniciamos carregando os valores de \$a0 e \$a1 em \$f0 e \$f2 respectivamente utilizando a função *ldc1* que carrega os valores no coprocessor 1

(registradores de ponto flutuante), após isso, com o mesmo processo, carregamos também, para trabalhar apenas com registradores de ponto flutuante, 2 em \$f4, carregando inicialmente o valor em \$t0, movendo o valor e convertendo de word para double. Fechamos então esta parte carregando \$t0 com \$a0 desta vez e iniciando \$t1 com 0 para instanciar o contador de 0 a n.

```
ldcl $f0, 0($a0) # estimativa
ldcl $f2, 0($a1) # x
## O MIPS nao permite carregar um inteiro direto em um reg float, precisa jogar em outro reg
# iniciando 4 constante 2 em $f4
li $t0, 2
mtcl $t0, $f4
cvt.d.w $f4, $f4

lw $t0, 0($a2) # n
li $t1, 0 # instanciando contador de 0 ate n
```

Figura 4 - Parte inicial raiz quadrada

Por fim, criamos um loop para executar a equação $Estimativa = \frac{(\frac{x}{Estimativa}) + Estimativa}{2}$, 'n' vezes seguindo a ordem $y(\$f6) = \frac{x(\$f2)}{estimativa(\$f0)}$, $y(\$f6) = y(\$f6) + estimativa(\$f0)$ e, por fim, $Estimativa(\$f0)_{nova} = \frac{y(\$f6)}{2(\$f4)}$.

Adicionando também 1 para servir como contador de quantidades e *blt* \$t1, \$t0, loop para retornar o loop sempre que \$t1 for menor ou igual a \$r0 finalizando com um *jr* \$ra para retornar à função da qual foi chamada.

```
loop:
div.d $f6, $f2, $f0 # f6 = x/est
add.d $f6, $f6, $f0 # f6 = x/est + est
div.d $f0, $f6, $f4 # estimativa

addi $t1, $t1, 1
blt $t1, $t0, loop

jr $ra
```

Figura 5 - Loop raiz_quadrada

C. Retornando para o main

Finalizando, retornamos ao main com o *jr* dentro do método iterativo de newton e executamos o procedimento das impressões no terminal. Começando movendo \$f0 para \$f12 e carregando 3 em \$v0, para mostrar o número obtido. Logo após o syscall, mudamos \$v0 para 4 enquanto o valor “espaco” é salvo dentro de \$a0 para que o sistema interprete sua requisição. Junto a isso, implementamos também, em double, a função *sqrt.d* a fim de comparar os resultados obtidos pelo

método de Newton com a própria função raiz quadrada. Por fim, carregamos 10 no \$v0 e utilizamos um último *Syscall* para fechar o sistema.

```
li $v0, 4
la $a0, espaco # Printando um espaco entre as duas respostas
syscall

li $v0, 3
sqrt.d $f12, $f2 # Calculo da raiz por uso da funcao
syscall

li $v0, 10
syscall
```

Figura 6 - Parte final main

III. CÁLCULO DO SENO POR SÉRIE

O proposto pela segunda etapa deste laboratório é a implementação do cálculo da função seno pela série da Equação 1, com a limitação de utilizar os primeiros 20 termos da série e sempre realizar os cálculos com os registradores de ponto flutuante.

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} \cdot x^{2n+1}$$

Equação 1 - Série de Taylor

Inicialmente, para a realização do cálculo, foram desenvolvidos dois procedimentos auxiliares, *factorial* e *power*.

A. Procedimento fatorial

Como o cálculo de fatoriais inteiros não é uma instrução do MIPS, foi necessário implementar um procedimento para realizar o exercício proposto. O procedimento *factorial* criado recebe um número inteiro em \$a0, e retorna o resultado no registrador \$f0. Vale reforçar que, por utilizar precisão dupla, o resultado também ocupa o registrador \$f1.

```
# Args, $a0: number
# Returns: $f0, result
factorial:
move $t0, $a0 # n
li $t1, 1
mtcl $t1, $f0
cvt.d.w $f0, $f0
beq $t0, $zero, factorial_end

factorial_loop:
mtcl $t0, $f10
cvt.d.w $f10, $f10

mul.d $f0, $f0, $f10

addi $t0, $t0, -1
bnez $t0, factorial_loop
```

Figura 7 - Procedimento factorial

O procedimento possui uma lógica simples, que inicia com a movimentação do argumento para a

utilização no coprocessador 1, a partir da instrução *mtc1* (move to coprocessor 1), e conversão de *word* para *double* com *cvt.d.w* (convert from word to double precision). Caso o número seja zero, o processo já é encerrado.

O loop do processo utiliza a instrução *mul.d* (multiply double), já que lidamos com precisão dupla. Também, como o coprocessador 1 não trabalha com imediatos, o controle do próximo número a ser calculado é feito com um inteiro no coprocessador 0, assim necessitando realizar a conversão a cada iteração. Uma solução válida para esse problema seria a instânciação do valor -1 em algum registrador de ponto flutuante para uso dentro dessa função.

Por fim, quando a variável de controle \$t0 atinge 0, o processo é terminado.

B. Procedimento de potência

A potenciação também é uma função não implementada nas funções padrão do MIPS. Portanto, foi criado o procedimento *power* para a calcular. Os argumentos recebidos são o endereço da base em \$a0 - que por decisão de arquitetura do programa já está salvo na memória, visto que é um dos argumentos do programa principal -, e o expoente em \$a1, que, por vez, é passado como *word*. A vantagem da utilização de um número na memória é que não necessita de operações de movimentação e conversão na lógica, podendo ser utilizado após um *ldc1* (load to coprocessor 1).

O processo em si funciona a partir de um loop que itera expoente vezes, multiplicando o valor da base por um acumulador que guarda as multiplicações anteriores.

```
# Args, $a0: numberAddr, $a1: exponent
# Returns: $f0, result
# Uses $t0-1, $f0-3
power:
    ldc1    $f2, 0($a0)    # float number
    move    $t0, $a1        # exponent

    # initializing acc 1
    li      $t1, 1
    mtc1    $t1, $f0
    cvt.d.w $f0, $f0

power_loop:
    mul.d   $f0, $f2, $f0    # acc = acc * x
    addi    $t0, $t0, -1

    bnez    $t0, power_loop
```

Figura 8 - Procedimento potenciação

C. Procedimento seno

Com os procedimentos auxiliares criados, é possível implementar o procedimento para o cálculo do seno a partir de uma aproximação por série. Os argumentos são o endereço do x em \$a0 e o número de iterações em \$a1. O retorno é um número de precisão dupla em \$f0.

Como esse é um procedimento não folha, e faz uso dos auxiliares descritos previamente, ele faz uso da stack para guardar a referência de onde foi chamado. Além disso, na sua inicialização, é criada uma constante -1 em \$f8, para posteriormente ser utilizado na conversão de sinal. A primeira iteração é somente x=x, então o loop de cálculo acontece 19 vezes.

```
# first iteration
# x = x
addi    $t2, $t2, -1

sin_loop:
    move    $a0, $s0        # setting x addr arg0
    move    $a1, $t3        # setting power arg1
    jal     power            # f0 = x^pow

    mov.d   $f12, $f0        # f12 = x^pow

    move    $a0, $t3        # setting factorial value
    jal     factorial

    mov.d   $f6, $f0         # $f6 = fat

    div.d   $f6, $f12, $f6    # f6 = x^pow/fat

    bgtz    $t4, positive

negative:
    mul.d   $f6, $f6, $f8

positive:
    add.d   $f4, $f4, $f6
```

Figura 9 - Loop do procedimento sin

A Figura 9 demonstra como são feitos os cálculos no procedimento, utilizando das instruções de ponto flutuante, no coprocessador 1. Dentro do procedimento, o controle se o próximo valor é positivo ou negativo para a adição ao acumulador é feito a partir de uma flag que inverte a cada iteração. Caso o valor seja positivo, o número obtido é somado diretamente. Caso contrário, ele é multiplicado por -1.

```
# variable update for next iteration
mul      $t4, $t4, -1    # inverting flag
addi     $t3, $t3, 2      # pow += 2
addi     $t2, $t2, -1    # iteration--
bnez     $t2, sin_loop
```

Figura 10 - Lógica das variáveis de controle

Por fim, o procedimento recupera o endereço de retorno da *stack*, e ao retornar pro main o número obtido é impresso no terminal.

D. Execução do programa

Como a coleta do parâmetro x não foi especificada nas premissas, seu valor é recebido por alteração direta no *.data*, com o valor de ângulo em radiano. Após a execução, o valor é apresentado no terminal. Para fins de teste, foram utilizados os valores e obtidos os resultados presentes na Tabela 1.

Valor de X (radianos)	Saída esperada	Saída obtida
0.0	0.0	0.0
1.570796	1.0	0,99999
0.523598	0,5	0.49999
-1.570796	-1	-0,9999
2.356194	~0.7071	0.70710

Tabela 1 - Resultados do programa

IV. RESULTADOS E LIMITAÇÕES

Apresentados os códigos durante o relatório, percebe-se que ambas premissas foram cumpridas com êxito, sendo alcançados os resultados esperados. Contudo, houveram algumas dificuldades principalmente relacionadas ao uso de dados e seus salvamentos em diferentes momentos do código, tornando assim algumas etapas talvez não tão ótimas quanto possível. Como exemplo temos a função *power*, que na nossa aplicação recebe um valor de endereço de memória para carregar o valor referente, em vez de receber apenas este valor diretamente. Quanto a limitações, diferente do laboratório anterior, não houveram problemas relacionados a isto, sendo possível escalar nossas implementações até o quanto o sistema permitir.